

---

## DBMS Cheat Sheet (Theory + Real-Life Analogies)

---

### 1. What is DBMS?

A **Database Management System** is software that lets you **store, retrieve, and manage data efficiently**.

**Analogy:** Think of a **digital filing cabinet** — it keeps data organized, searchable, and secure.

---

### 2. Types of DBMS

| Type              | Example              | Use Case                            |
|-------------------|----------------------|-------------------------------------|
| Hierarchical      | IBM IMS              | Org charts, XML parsing             |
| Network           | IDMS                 | Complex many-to-many relationships  |
| <b>Relational</b> | <b>MySQL, Oracle</b> | Most popular: tables + SQL          |
| NoSQL             | MongoDB, Cassandra   | Unstructured/big data, JSON storage |

---

### 3. Relational Model (RDBMS)

- **Table = Relation**
  - **Row = Tuple**
  - **Column = Attribute**
  - Tables are related via **keys**.
- 

### 4. Keys in DBMS

| Key Type             | Role                                    |
|----------------------|-----------------------------------------|
| <b>Primary Key</b>   | Uniquely identifies each row (e.g., ID) |
| <b>Candidate Key</b> | All possible unique identifiers         |
| <b>Super Key</b>     | Primary Key + extra attributes          |

| Key Type             | Role                                                        |
|----------------------|-------------------------------------------------------------|
| <b>Foreign Key</b>   | Refers to a primary key in another table (creates relation) |
| <b>Composite Key</b> | Primary key made of multiple columns                        |

Real-life: A primary key is like your **Aadhar number** — unique and non-null.

---

## 5. SQL Categories

| SQL Type          | Commands                      |
|-------------------|-------------------------------|
| DDL (Define)      | CREATE, ALTER, DROP, TRUNCATE |
| DML (Manipulate)  | INSERT, UPDATE, DELETE        |
| DQL (Query)       | SELECT                        |
| DCL (Control)     | GRANT, REVOKE                 |
| TCL (Transaction) | COMMIT, ROLLBACK, SAVEPOINT   |

---

## 6. Normalization (Remove Redundancy)

- **1NF**: Atomic values (no arrays/lists)
- **2NF**: Remove partial dependency (for composite keys)
- **3NF**: Remove transitive dependency
- **BCNF**: Stronger version of 3NF

Analogy: Like cleaning a fridge — remove duplicates, categorize food, and label it properly.

---

## 7. Joins in SQL

| Join Type  | Description              |
|------------|--------------------------|
| INNER JOIN | Matching rows only       |
| LEFT JOIN  | All left + matched right |
| RIGHT JOIN | All right + matched left |

| Join Type | Description |
|-----------|-------------|
|-----------|-------------|

|            |                                            |
|------------|--------------------------------------------|
| FULL OUTER | All rows from both, matched where possible |
|------------|--------------------------------------------|

|           |                                             |
|-----------|---------------------------------------------|
| SELF JOIN | Table joins itself (e.g., manager-employee) |
|-----------|---------------------------------------------|

---

## 8. Transactions & ACID Properties

### Letter Meaning

- A     Atomicity — all or nothing
- C     Consistency — data remains valid
- I     Isolation — transactions don't interfere
- D     Durability — changes are permanent

Real-life: Like transferring money between two accounts — either both debit & credit happen, or neither.

---

## 9. Indexes

- Improves **search speed**
- Like a **book index** — jumps directly to data.

```
CREATE INDEX idx_name ON Employee(Name);
```

---

## 10. Constraints

- Enforce rules on data:
    - NOT NULL
    - UNIQUE
    - PRIMARY KEY
    - FOREIGN KEY
    - CHECK (e.g., Age > 18)
    - DEFAULT
-

## 🔗 11. Difference Between

| Concept   | RDBMS                            | NoSQL                    |
|-----------|----------------------------------|--------------------------|
| Structure | Tables                           | JSON / Key-Value / Graph |
| Schema    | Fixed                            | Flexible                 |
| Scaling   | Vertical                         | Horizontal               |
| Examples  | MySQL, Oracle MongoDB, Cassandra |                          |

---

## 🌈 12. ER Diagram Basics

- Entity = Object (e.g., Student)
- Attribute = Properties (e.g., Name, Age)
- Relationship = Connection (e.g., EnrolledIn)

Draw using rectangles (entities), ovals (attributes), diamonds (relationships)

---

concise summary of **Codd's 12 Rules** — foundational principles for relational database systems, proposed by **Dr. E.F. Codd**, the father of the relational model.

Think of them as a "**litmus test**" to determine if a DBMS is truly **relational**.

---

### Codd's 12 Rules (0 to 12)

| Rule No. | Name                              | Description (In Simple Terms)                                                                                                 |
|----------|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Rule 0   | Foundation Rule                   | A DBMS must be able to manage data using only relational capabilities to be considered a relational DBMS.                     |
| Rule 1   | Information Rule                  | All data is stored in tables (relations), and everything is represented in <b>rows and columns</b> .                          |
| Rule 2   | Guaranteed Access Rule            | Every piece of data (atomic value) must be accessible by table name, column name, and primary key — no pointer or navigation. |
| Rule 3   | Systematic Treatment of Nulls     | Nulls (missing/unknown values) must be handled <b>consistently</b> , distinct from 0 or blank.                                |
| Rule 4   | Dynamic Online Catalog            | Database structure (schema) is stored in tables just like user data and can be queried using SQL.                             |
| Rule 5   | Comprehensive Data Sub-language   | There must be a complete language like <b>SQL</b> for querying, defining, updating, and managing data.                        |
| Rule 6   | View Updating Rule                | Views (virtual tables) must be <b>logically updatable</b> if the base tables support it.                                      |
| Rule 7   | High-Level Insert, Update, Delete | The DBMS should support <b>set-level</b> operations — not just one row at a time.                                             |
| Rule 8   | Physical Data Independence        | Changes in how data is stored (e.g., on disk) <b>should not affect</b> application logic.                                     |
| Rule 9   | Logical Data Independence         | Changes to table structure (adding/removing columns) <b>should not break</b> existing applications.                           |

| Rule No. | Name                             | Description (In Simple Terms)                                                                                        |
|----------|----------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Rule 10  | <b>Integrity Independence</b>    | Integrity constraints (e.g., PK, FK) should be stored in the catalog and enforced by the DBMS — not in the app code. |
| Rule 11  | <b>Distribution Independence</b> | Users should not need to know whether the data is local or distributed.                                              |
| Rule 12  | <b>Non-Subversion Rule</b>       | Low-level access methods (e.g., file-level) must not bypass the relational model or security.                        |

---

 **Mnemonic to Remember:**

**"Every Good Student Can Read Very High-Level Math, Especially If Learning DBMS Concepts."**

(Each capitalized word maps to a rule.)

---

 **Quick Analogy**

Imagine a library:

- **Bookshelves = Tables**
  - **Book = Row**
  - **Column = Title, Author, etc.**
  - You should be able to find a book by its title or ISBN (primary key), even if the physical arrangement changes — that's Codd's idea in practice.
-

**Normalization in DBMS**, explained with **theory, logic, real-life analogy, and examples** — perfect for interviews, exams, or project work.

---

## What is Normalization?

**Normalization** is the process of organizing data in a database to:

- **Eliminate redundancy**
  - **Avoid anomalies** (insertion, update, deletion)
  - **Ensure data integrity**
- 

## Real-Life Analogy:

Imagine storing books in a library:

- **Unnormalized:** Same book appears in multiple places.
  - **Normalized:** Each book is listed once, with clear links to authors, genres, etc.
- 

## Forms of Normalization (1NF to 5NF)

---

### 1NF (First Normal Form) – Atomicity

**Rule:** Each column should have **atomic (indivisible)** values. No arrays or multiple values in a single cell.

**Example (Before 1NF):**

#### **Student Subjects**

Alice     Math, Science

**After 1NF:**

#### **Student Subject**

Alice     Math

Alice     Science

---

## ✅ 2NF (Second Normal Form) – Remove Partial Dependency

**Rule:** Must be in 1NF and all **non-key attributes** must depend on **whole primary key** (not just part of it).

Applies mainly to **composite keys**.

### **Example:**

If StudentID + CourseID is the primary key, then StudentName should not be dependent only on StudentID.

---

## ✅ 3NF (Third Normal Form) – Remove Transitive Dependency

**Rule:** Must be in 2NF and non-key attributes should **not depend on other non-key attributes**.

“Non-key should not depend on non-key.”

### **Example (Violation of 3NF):**

**EmpID EmpName DeptID DeptName**

Here, DeptName depends on DeptID → **transitive dependency**.  
Split into separate tables.

---

## ✅ BCNF (Boyce-Codd Normal Form) – Strong 3NF

**Rule:** For every **functional dependency** ( $A \rightarrow B$ ), A should be a **super key**.

Handles edge cases not solved by 3NF.

**Use When:** You have multiple candidate keys and a non-prime attribute depends on the wrong key.

---

## ✅ 4NF – Multivalued Dependency Removal

**Rule:** Must be in BCNF and no **multivalued dependencies**.

**Example:** If a student can have **multiple emails and multiple phone numbers**, store those in separate tables.

---

## ✅ 5NF – Join Dependency

**Rule:** Must be in 4NF and **no lossless decomposition** when splitting/joining multiple tables.

Very rare in practice.

---

### ! Why Normalize?

| Problem | Example |
|---------|---------|
|---------|---------|

|            |                                     |
|------------|-------------------------------------|
| Redundancy | Same data repeated in multiple rows |
|------------|-------------------------------------|

|                |                                    |
|----------------|------------------------------------|
| Update Anomaly | Need to update data in many places |
|----------------|------------------------------------|

|                |                                              |
|----------------|----------------------------------------------|
| Insert Anomaly | Can't add a student unless a course is added |
|----------------|----------------------------------------------|

|                |                                             |
|----------------|---------------------------------------------|
| Delete Anomaly | Deleting one item may delete essential data |
|----------------|---------------------------------------------|

---

### Quick Summary

| Form | Purpose                   | Fixes                   |
|------|---------------------------|-------------------------|
| 1NF  | Atomic values only        | Repeating groups        |
| 2NF  | Full dependency on PK     | Partial dependency      |
| 3NF  | No transitive dependency  | Transitive dependency   |
| BCNF | Stronger key rules        | All anomalies of 3NF    |
| 4NF  | No multivalued dependency | Independent multivalues |
| 5NF  | No join dependency        | Complex decompositions  |

---

---

## SQL Cheat Sheet — All-In-One

---

### 1. SQL Types (By Purpose)

| Category Full Form |                     | Commands                      |
|--------------------|---------------------|-------------------------------|
| DDL                | Data Definition     | CREATE, ALTER, DROP, TRUNCATE |
| DML                | Data Manipulation   | INSERT, UPDATE, DELETE        |
| DQL                | Data Query          | SELECT                        |
| DCL                | Data Control        | GRANT, REVOKE                 |
| TCL                | Transaction Control | COMMIT, ROLLBACK, SAVEPOINT   |

---

### 2. DDL – Define Structure

```
CREATE TABLE Employee (  
    EmpID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Salary DECIMAL(10,2)  
);
```

```
ALTER TABLE Employee ADD Dept VARCHAR(20);
```

```
DROP TABLE Employee;
```

```
TRUNCATE TABLE Employee; -- Deletes all rows but keeps structure
```

---

### 3. DML – Manage Data

```
INSERT INTO Employee (EmpID, Name, Salary) VALUES (101, 'John', 50000);
```

```
UPDATE Employee SET Salary = 55000 WHERE EmpID = 101;
```

```
DELETE FROM Employee WHERE EmpID = 101;
```

---

#### 4. DQL – Query Data

SELECT \* FROM Employee;

SELECT Name, Salary FROM Employee WHERE Salary > 50000;

SELECT COUNT(\*) FROM Employee WHERE Dept = 'IT';

---

#### 5. Joins – Combine Tables

| Join Type | Description |
|-----------|-------------|
|-----------|-------------|

|            |                    |
|------------|--------------------|
| INNER JOIN | Only matching rows |
|------------|--------------------|

|           |                           |
|-----------|---------------------------|
| LEFT JOIN | All left + matching right |
|-----------|---------------------------|

|            |                           |
|------------|---------------------------|
| RIGHT JOIN | All right + matching left |
|------------|---------------------------|

|           |                                     |
|-----------|-------------------------------------|
| FULL JOIN | All rows, matched wherever possible |
|-----------|-------------------------------------|

SELECT E.Name, D.DeptName

FROM Employee E

INNER JOIN Department D ON E.DeptID = D.DeptID;

---

#### 6. Constraints – Rules on Columns

| Constraint | Purpose |
|------------|---------|
|------------|---------|

|             |                           |
|-------------|---------------------------|
| PRIMARY KEY | Uniquely identifies a row |
|-------------|---------------------------|

|             |                        |
|-------------|------------------------|
| FOREIGN KEY | Links to another table |
|-------------|------------------------|

|        |                     |
|--------|---------------------|
| UNIQUE | No duplicate values |
|--------|---------------------|

|          |                   |
|----------|-------------------|
| NOT NULL | Must have a value |
|----------|-------------------|

|         |                             |
|---------|-----------------------------|
| DEFAULT | Auto-fill if no value given |
|---------|-----------------------------|

|       |                                  |
|-------|----------------------------------|
| CHECK | Custom condition (e.g. Age > 18) |
|-------|----------------------------------|

---

#### 7. Subqueries – Query Inside a Query

-- Employees with salary above average

```
SELECT Name FROM Employee  
WHERE Salary > (SELECT AVG(Salary) FROM Employee);
```

---

## 8. Aggregate Functions

### Function Use

COUNT() Number of rows

SUM() Total of values

AVG() Average value

MIN() Smallest value

MAX() Largest value

```
SELECT COUNT(*) FROM Employee;
```

```
SELECT AVG(Salary) FROM Employee;
```

---

## 9. GROUP BY & HAVING

```
SELECT Dept, AVG(Salary)
```

```
FROM Employee
```

```
GROUP BY Dept
```

```
HAVING AVG(Salary) > 50000;
```

---

## 10. Set Operators

### Operator Description

UNION Combine results (unique)

UNION ALL Combine all (with duplicates)

INTERSECT Common rows

EXCEPT Rows in A but not B

---

## 11. Transactions (TCL)

```
BEGIN;  
  
UPDATE Employee SET Salary = Salary + 5000;  
  
COMMIT;  
  
-- or rollback  
  
ROLLBACK;
```

---

## 12. Indexes – Speed Up Queries

```
CREATE INDEX idx_name ON Employee(Name);  
  
DROP INDEX idx_name;
```

---

## 13. Views – Virtual Tables

```
CREATE VIEW HighEarnings AS  
  
SELECT Name, Salary FROM Employee WHERE Salary > 80000;
```

---

## 14. Stored Procedure (Reusable Logic)

```
CREATE PROCEDURE GetHighEarnings()  
  
AS  
  
BEGIN  
  
    SELECT * FROM Employee WHERE Salary > 80000;  
  
END;
```

---

## 15. Basic Query Tips

- Always use **WHERE** to filter early
- Use **LIMIT** to avoid large result sets
- Prefer **INNER JOIN** for matching data

- Use **GROUP BY** only when using aggregates
  - Always **normalize data** to reduce redundancy
- 

## ✓ 1. Table Creation

### 🎓 Student Table

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Gender VARCHAR(10),  
    Age INT,  
    City VARCHAR(50)  
);
```

### 📖 Education Table

```
CREATE TABLE Education (  
    EduID INT PRIMARY KEY,  
    StudentID INT,  
    Degree VARCHAR(50),  
    Percentage DECIMAL(5,2),  
    Year INT,  
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID)  
);
```

---

## ✓ 2. Sample Data

### Student Entries:

```
INSERT INTO Student VALUES  
(1, 'Alice', 'Female', 21, 'Delhi'),  
(2, 'Bob', 'Male', 22, 'Mumbai'),  
(3, 'Charlie', 'Male', 23, 'Delhi'),
```

(4, 'David', 'Male', 22, 'Bangalore'),  
(5, 'Eve', 'Female', 21, 'Hyderabad'),  
(6, 'Frank', 'Male', 24, 'Delhi'),  
(7, 'Grace', 'Female', 23, 'Mumbai'),  
(8, 'Heidi', 'Female', 22, 'Bangalore'),  
(9, 'Ivan', 'Male', 21, 'Hyderabad'),  
(10, 'Judy', 'Female', 23, 'Delhi');

#### **Education Entries:**

INSERT INTO Education VALUES

(101, 1, 'B.Tech', 88.50, 2023),  
(102, 2, 'B.Sc', 76.20, 2022),  
(103, 3, 'B.Com', 65.00, 2023),  
(104, 4, 'B.Tech', 92.00, 2022),  
(105, 5, 'B.Sc', 59.00, 2023),  
(106, 6, 'BCA', 80.00, 2021),  
(107, 7, 'B.Tech', 95.00, 2023),  
(108, 8, 'B.Sc', 78.00, 2022),  
(109, 9, 'B.Com', 82.00, 2021),  
(110, 10, 'B.Tech', 68.00, 2023);

---

### **3. Most Important Queries**

---

#### **A. INNER JOIN (Get student info with degree and percentage)**

```
SELECT s.Name, s.City, e.Degree, e.Percentage
FROM Student s
INNER JOIN Education e ON s.StudentID = e.StudentID;
```

---

#### **B. LEFT JOIN (Students who may not have education data)**

```
SELECT s.Name, e.Degree
FROM Student s
LEFT JOIN Education e ON s.StudentID = e.StudentID;
```

---

#### **C. GROUP BY + Aggregate (Avg % by Degree)**

```
SELECT Degree, AVG(Percentage) AS Avg_Percentage
FROM Education
GROUP BY Degree;
```

---

#### **D. GROUP BY + HAVING (Only degrees with average > 80%)**

```
SELECT Degree, AVG(Percentage) AS Avg_Perc
FROM Education
GROUP BY Degree
HAVING AVG(Percentage) > 80;
```

---

#### **E. Nested Query – Students with % > avg**

```
SELECT Name
FROM Student s
JOIN Education e ON s.StudentID = e.StudentID
WHERE e.Percentage > (
    SELECT AVG(Percentage) FROM Education
);
```

---

#### **F. Top 3 Students by Percentage**

```
SELECT s.Name, e.Percentage
FROM Student s
JOIN Education e ON s.StudentID = e.StudentID
ORDER BY e.Percentage DESC
```

LIMIT 3;

---

### **G. Count Students per City**

```
SELECT City, COUNT(*) AS Total_Students
FROM Student
GROUP BY City;
```

---

### **H. Using CHECK & NOT NULL for Data Quality**

```
ALTER TABLE Education
ADD CHECK (Percentage >= 0 AND Percentage <= 100);
```

---

### **Tips & Tricks for Writing SQL Queries**

| Tip                     | Description                                                                   |
|-------------------------|-------------------------------------------------------------------------------|
| Use JOIN wisely         | Prefer INNER JOIN for matching rows. Use LEFT JOIN to include unmatched rows. |
| Always use WHERE        | To filter early and avoid scanning all rows                                   |
| GROUP BY needs HAVING   | Use HAVING only after GROUP BY, not WHERE                                     |
| Use aliases             | Shorten table names using aliases like s, e for readability                   |
| Index important columns | Especially ones used in joins or WHERE clauses                                |
| Avoid SELECT *          | It fetches unnecessary data — select only needed columns                      |
| Use LIMIT               | To test queries with fewer results                                            |
| Nest smartly            | Use subqueries only when needed; prefer joins for performance                 |
| Format nicely           | Use indentation, line breaks for readability in complex queries               |

---

Let's create **2 realistic tables**: Student and Education with 10 entries each. Then, we'll explore **important SQL queries**, including JOINS, GROUP BY, HAVING, **nested queries**, and **tips/tricks** for writing better queries.

---

## ✅ 1. Table Creation

### 🎓 Student Table

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Gender VARCHAR(10),  
    Age INT,  
    City VARCHAR(50)  
);
```

### 📖 Education Table

```
CREATE TABLE Education (  
    EduID INT PRIMARY KEY,  
    StudentID INT,  
    Degree VARCHAR(50),  
    Percentage DECIMAL(5,2),  
    Year INT,  
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID)  
);
```

---

## ✅ 2. Sample Data

### Student Entries:

```
INSERT INTO Student VALUES  
(1, 'Alice', 'Female', 21, 'Delhi'),  
(2, 'Bob', 'Male', 22, 'Mumbai'),
```

(3, 'Charlie', 'Male', 23, 'Delhi'),  
(4, 'David', 'Male', 22, 'Bangalore'),  
(5, 'Eve', 'Female', 21, 'Hyderabad'),  
(6, 'Frank', 'Male', 24, 'Delhi'),  
(7, 'Grace', 'Female', 23, 'Mumbai'),  
(8, 'Heidi', 'Female', 22, 'Bangalore'),  
(9, 'Ivan', 'Male', 21, 'Hyderabad'),  
(10, 'Judy', 'Female', 23, 'Delhi');

#### **Education Entries:**

INSERT INTO Education VALUES

(101, 1, 'B.Tech', 88.50, 2023),  
(102, 2, 'B.Sc', 76.20, 2022),  
(103, 3, 'B.Com', 65.00, 2023),  
(104, 4, 'B.Tech', 92.00, 2022),  
(105, 5, 'B.Sc', 59.00, 2023),  
(106, 6, 'BCA', 80.00, 2021),  
(107, 7, 'B.Tech', 95.00, 2023),  
(108, 8, 'B.Sc', 78.00, 2022),  
(109, 9, 'B.Com', 82.00, 2021),  
(110, 10, 'B.Tech', 68.00, 2023);



#### **A. INNER JOIN (Get student info with degree and percentage)**

```
SELECT s.Name, s.City, e.Degree, e.Percentage  
FROM Student s  
INNER JOIN Education e ON s.StudentID = e.StudentID;
```



#### **B. LEFT JOIN (Students who may not have education data)**

```
SELECT s.Name, e.Degree
```

```
FROM Student s
LEFT JOIN Education e ON s.StudentID = e.StudentID;
```

---

### C. GROUP BY + Aggregate (Avg % by Degree)

```
SELECT Degree, AVG(Percentage) AS Avg_Percentage
FROM Education
GROUP BY Degree;
```

---

### D. GROUP BY + HAVING (Only degrees with average > 80%)

```
SELECT Degree, AVG(Percentage) AS Avg_Perc
FROM Education
GROUP BY Degree
HAVING AVG(Percentage) > 80;
```

---

### E. Nested Query – Students with % > avg

```
SELECT Name
FROM Student s
JOIN Education e ON s.StudentID = e.StudentID
WHERE e.Percentage > (
    SELECT AVG(Percentage) FROM Education
);
```

---

### F. Top 3 Students by Percentage

```
SELECT s.Name, e.Percentage
FROM Student s
JOIN Education e ON s.StudentID = e.StudentID
ORDER BY e.Percentage DESC
LIMIT 3;
```

---

### G. Count Students per City

```
SELECT City, COUNT(*) AS Total_Students
FROM Student
GROUP BY City;
```

---

### H. Using CHECK & NOT NULL for Data Quality

```
ALTER TABLE Education
ADD CHECK (Percentage >= 0 AND Percentage <= 100);
```

---

### Tips & Tricks for Writing SQL Queries

| Tip                     | Description                                                                   |
|-------------------------|-------------------------------------------------------------------------------|
| Use JOIN wisely         | Prefer INNER JOIN for matching rows. Use LEFT JOIN to include unmatched rows. |
| Always use WHERE        | To filter early and avoid scanning all rows                                   |
| GROUP BY needs HAVING   | Use HAVING only after GROUP BY, not WHERE                                     |
| Use aliases             | Shorten table names using aliases like s, e for readability                   |
| Index important columns | Especially ones used in joins or WHERE clauses                                |
| Avoid SELECT *          | It fetches unnecessary data — select only needed columns                      |
| Use LIMIT               | To test queries with fewer results                                            |
| Nest smartly            | Use subqueries only when needed; prefer joins for performance                 |
| Format nicely           | Use indentation, line breaks for readability in complex queries               |

---

Great! Let's dive into a **complete advanced SQL query** that includes:

- SELECT

- FROM
  - JOIN
  - WHERE
  - GROUP BY
  - HAVING
  - **Nested subqueries** in SELECT, FROM, and WHERE
  - **All best practices**
  - **Rules/syntax explanation** step by step
- 

Show the top 2 degrees (by average percentage) that have students from Delhi with above-average performance compared to their own degree.

---

### **Complex Query (with nested subqueries)**

```
SELECT Degree, AVG(Percentage) AS Avg_Per_Degree
```

```
FROM (
```

```
    SELECT s.Name, s.City, e.Degree, e.Percentage
```

```
    FROM Student s
```

```
    INNER JOIN Education e ON s.StudentID = e.StudentID
```

```
    WHERE s.City = 'Delhi'
```

```
    AND e.Percentage > (
```

```
        SELECT AVG(Percentage)
```

```
        FROM Education e2
```

```
        WHERE e2.Degree = e.Degree
```

```
    )
```

```
) AS DelhiAboveAvg
```

```
GROUP BY Degree
```

```
HAVING AVG(Percentage) > 70
```

```
ORDER BY Avg_Per_Degree DESC
```

LIMIT 2;

---

## Breakdown + Rulebook

---

### 1. SELECT Clause

- Used to **choose columns** or **aggregated data**.
- Can include **expressions, functions,** and **nested subqueries** (but be careful with performance).

SELECT column1, column2, AVG(column3)

✓ **Best Practice:** Avoid SELECT \* in production.

---

### 2. FROM Clause

- This is where the **main table or subquery** comes from.
- You **can use a nested subquery here** — this is called a **derived table** or **inline view**.

FROM (SELECT ...) AS alias

✓ **Best Practice:** Always use an alias when using a subquery in FROM.

---

### 3. JOIN Clause

- Used to combine rows from multiple tables.
- INNER JOIN: Only matched rows.
- LEFT JOIN: All from left + matched from right.
- RIGHT JOIN: All from right + matched from left.
- FULL JOIN: Everything matched or unmatched.

FROM A

INNER JOIN B ON A.id = B.id

✓ Use **JOINS** instead of subqueries where possible for better performance.

---

#### 4. WHERE Clause

- Used to **filter rows before grouping or aggregation**.
- You **can use nested subqueries** here.

WHERE column > (SELECT ...)

✓ Use **WHERE** to reduce rows early and speed up queries.

---

#### 5. GROUP BY Clause

- Used when you apply aggregate functions (like AVG, SUM, COUNT) and want results **grouped** by columns.

GROUP BY Degree

✓ **Must use with aggregates** in SELECT clause.

---

#### 6. HAVING Clause

- Similar to WHERE but applies **after GROUP BY**, used to **filter aggregated groups**.

HAVING AVG(Salary) > 70000

✓ Use HAVING when filtering **aggregated results** (can't use WHERE here).

---

#### 7. ORDER BY Clause

- Used to sort result set based on one or more columns or expressions.

ORDER BY column DESC

✓ Always comes **after GROUP BY and HAVING**.

---

#### 8. LIMIT Clause

- Used to restrict number of rows returned.

LIMIT 5

✓ Helps test and limit output.

---

### 💡 When and Where to Use Subqueries:

| Location  | Example Use                                | Notes                               |
|-----------|--------------------------------------------|-------------------------------------|
| In SELECT | (SELECT COUNT(*) FROM Education)           | Scalar subquery (returns 1 value)   |
| In FROM   | (SELECT ...) AS alias                      | Derived table — behaves like a view |
| In WHERE  | WHERE Age > (SELECT AVG(Age) FROM Student) | Filter using calculated criteria    |
| In HAVING | HAVING AVG(Salary) > (SELECT ...)          | Compare aggregated result           |

✅ Prefer **JOINS** over **nested subqueries in SELECT** for performance.

---

### 🧠 Key SQL Writing Order (Mental Model)

💬 Think like SQL does:

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

#### Why?

- SQL evaluates **FROM** first to get data source.
  - Then **filters** it using WHERE.
  - Then **groups** it with GROUP BY.
  - Then filters **aggregated groups** via HAVING.
  - Then **selects** columns.
  - Then **sorts** using ORDER BY.
  - Finally, **limits** with LIMIT.
- 

Here's a **complete SQL theory notes** collection covering:

- **Data Models**
- **ACID Properties**
- **Transactions**

- **Triggers**
- **Stored Procedures**

Everything is provided in **point-wise format** for easy memorization and revision.

---

## **1. Data Models in DBMS**

A **Data Model** defines how data is connected, stored, and accessed.

### ◆ **Types of Data Models:**

#### **1. Hierarchical Model**

- Data organized in tree-like structure (parent-child).
- One parent can have multiple children; child has only one parent.
- Fast for one-to-many relationships.
- Example: XML, Windows Registry.

#### **2. Network Model**

- Uses graph structure with multiple parent-child relationships.
- Many-to-many relationships allowed.
- Data accessed using **pointers**.
- Example: CODASYL DBMS.

#### **3. Relational Model**

- Data stored in **tables (relations)**.
- Based on **E.F. Codd's rules**.
- Uses **primary key, foreign key**, etc.
- Structured Query Language (SQL) is used.
- Example: MySQL, PostgreSQL, Oracle.

#### **4. Entity-Relationship (E-R) Model**

- High-level conceptual model.
- Defines entities, attributes, and relationships.
- Used for **database design**.

#### **5. Object-Oriented Model**

- Combines object-oriented programming concepts.
- Stores objects directly in the DB.
- Example: db4o, ObjectDB.

---

## 2. ACID Properties

**ACID** ensures safe and reliable transactions in DBMS.

| Property               | Description                                                                           |
|------------------------|---------------------------------------------------------------------------------------|
| <b>A - Atomicity</b>   | "All or nothing". A transaction either completes fully or not at all.                 |
| <b>C - Consistency</b> | Database moves from one valid state to another; all rules (constraints) are followed. |
| <b>I - Isolation</b>   | Transactions are isolated from each other; one doesn't affect the other's data.       |
| <b>D - Durability</b>  | Once a transaction is committed, changes are permanent, even if the system crashes.   |

---

## 3. Transactions in SQL

A **transaction** is a group of operations performed as a single unit.

### ◆ States of a Transaction:

1. **Active** – Executing.
2. **Partially Committed** – Final operation done.
3. **Committed** – Changes saved.
4. **Failed** – Error occurred.
5. **Aborted** – Rolled back to previous state.

### ◆ Control Commands:

- BEGIN TRANSACTION – Start
- COMMIT – Save changes
- ROLLBACK – Undo changes
- SAVEPOINT – Mark a point in transaction to rollback partially

---

## 4. Triggers

A **Trigger** is a block of SQL code that automatically runs when a specific DB event occurs.

### ◆ When to Use:

- Enforce business rules
- Auto-log changes
- Validate inputs
- Create audit trails

### ◆ Trigger Events:

- BEFORE INSERT
- AFTER INSERT
- BEFORE UPDATE
- AFTER DELETE, etc.

### ◆ Key Features:

- Can't be manually called.
- Can access OLD and NEW values.
- Tied to a **table**.

```
CREATE TRIGGER trg_before_insert
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
    -- your logic
END;
```

---

## 5. Stored Procedures

A **Stored Procedure** is a precompiled set of SQL statements stored in the database.

### ◆ Advantages:

- Reusability
- Performance (compiled once)
- Security (controlled access)
- Reduces network traffic

◆ **Syntax:**

```
CREATE PROCEDURE GetEmployeeDetails (IN emplId INT)
```

```
BEGIN
```

```
    SELECT * FROM Employee WHERE ID = emplId;
```

```
END;
```

◆ **Types of Parameters:**

- IN: input only
- OUT: output only
- INOUT: both input and output

◆ **Use Cases:**

- Data validation
- Business logic
- Batch operations
- Reporting

---

 **BONUS: Carry-Lookahead Concept (In SQL Logic)**

**Carry-Lookahead** is not part of SQL — it's a **digital electronics concept** used in fast adders. But if you're referring to **logical optimization in queries**, here are equivalent ideas:

- **Use indexes** to skip full scans (like jumping ahead)
  - **Query optimization plans** (RDBMS engine decides best path)
  - **Avoid nested loops in joins**, use **hash or merge joins** if possible
  - Think in terms of **minimizing read blocks**, similar to reducing carry propagation
-

## Summary Sheet (High-Level)

| Topic | Key Point |
|-------|-----------|
|-------|-----------|

|             |                                                        |
|-------------|--------------------------------------------------------|
| Data Models | Hierarchical, Network, Relational, ER, Object-Oriented |
|-------------|--------------------------------------------------------|

|      |                                               |
|------|-----------------------------------------------|
| ACID | Atomicity, Consistency, Isolation, Durability |
|------|-----------------------------------------------|

|              |                                    |
|--------------|------------------------------------|
| Transactions | BEGIN, COMMIT, ROLLBACK, SAVEPOINT |
|--------------|------------------------------------|

|          |                                   |
|----------|-----------------------------------|
| Triggers | Auto-fire on INSERT/UPDATE/DELETE |
|----------|-----------------------------------|

|            |                                              |
|------------|----------------------------------------------|
| Procedures | Stored routines with IN/OUT/INOUT parameters |
|------------|----------------------------------------------|

---